Mixed Reality for Autonomous Racing

Melissa Dietsche
ETH Zürich
mdietsche@ethz.ch

Tae Kim
ETH Zürich
taekim@ethz.ch

Martin Defuns
ETH Zürich
mdefuns@ethz.ch

Oliver Šaravanja
ETH Zürich
osaravanja@ethz.ch

Abstract

Understanding and debugging autonomous vehicle behavior during operation remains challenging as internal decision-making processes are difficult to observe in real-time. This work presents a Mixed Reality visualization system for the Chronos autonomous racing platform. Building upon the teleoperation framework by Wilder-Smith et al., the system streams real-time vehicle data from the Chronos Racing System (CRS) via ROS, enabling spatial overlay of internal states onto the physical environment.

To facilitate debugging, we implemented six core visualization features including track alignment, control input displays, trajectory predictions, state trails, and vehicle state estimation. The modular architecture extends the existing framework to enable future additions. We conducted A/B testing with five Chronos developers evaluating track alignment methods (automatic vs. manual) and visualization approaches (fixed on HUD vs. car-following).

Results revealed that effective MR debugging interfaces require context-aware hybrid workflows rather than binary design choices. Developers unanimously preferred combining automatic initialization with manual refinement for alignment, revealing how one-time setup costs differ from repeated-use performance. Visualization preferences were task-dependent: 67% preferred fixed HUD for detailed parameter monitoring, while recognizing car-following displays for attention-focused observation. The system achieved strong ratings for behavior understanding (4.33/5) and debugging utility (3.83/5). The implementation includes comprehensive setup documentation and tutorials facilitating future development by the Chronos team.

1. Introduction

Autonomous racing platforms like CRS [2] enable researchers and students to develop control algorithms in accessible, cost-effective settings. However, a persistent challenge is understanding what the vehicle "thinks" during operation. When debugging trajectory planning or control behavior, developers typically rely on post-hoc log analysis, reviewing data files after runs complete. This separation between execution and analysis makes it difficult to build intuition about controller behavior.

Mixed Reality (MR) offers a different approach. By overlaying the vehicle's internal state directly onto the physical environment, MR enables developers to see predicted trajectories, planned paths, and control inputs spatially registered with the moving car [6]. This spatial co-location can transform debugging from a retrospective analysis task into an intuitive, real-time observation process. Building upon the teleoperation framework by Wilder-Smith et al. [6], we developed a MR visualization system specifically tailored for the Chronos racing platform, streaming real-time vehicle data from CRS via ROS-TCP-Endpoint to a Meta Quest 3 headset.

Designing effective MR interfaces for autonomous vehicle debugging requires addressing several practical challenges. First, virtual content must be precisely aligned with the physical environment, should this alignment be automatic, manual, or hybrid? Second, control inputs and vehicle state must be displayed during high-speed motion, should information follow the vehicle or remain fixed in the user's view? Finally, which design choices actually support real-world debugging workflows versus theoretical preferences? This report explains our approach to develop such MR interface and addresses these questions through evaluation of an MR visualization system for the CRS platform.

Section 2 explains the implemented visualization features. Section 3 details our A/B testing methodology comparing alignment and visualization approaches, and Section 4 presents quantitative and qualitative insights from developer testing. Section 5 presents the final refined implementations based on user feedback and section 6 discusses limitations and future work.

2. Features

2.1. Tools and Framework

Our mixed reality application builds upon several key software components to enable real-time visualization of autonomous racing data. As the foundation for our mixed re-

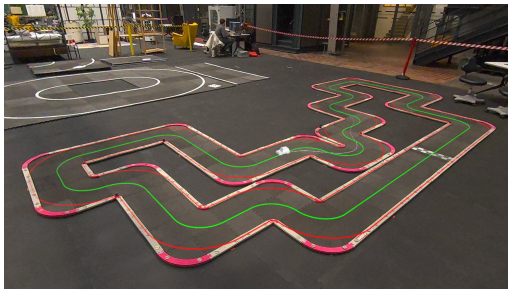


Figure 1. Virtual track aligned onto the physical Chronos racing track at the ETH ML Hall. The track boundary (red) and center line (green) are overlaid on the real track, allowing users to visualize the racing environment in mixed reality.

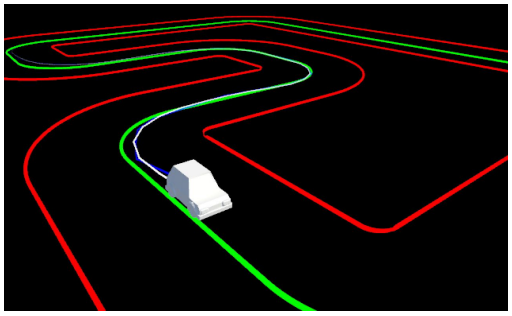


Figure 2. Trails for both ground truth state (white) and estimated state (blue) allow for better comparison between the two.

ality visualization, we extend the teleoperation framework developed by [6], which provides the core infrastructure for remote vehicle monitoring. The vehicle dynamics and sensor data are sourced from the CRS platform [2], which provides both a simulation environment and a physical autonomous racing vehicle, allowing our application to operate with either data source interchangeably. A central challenge in developing mixed reality applications that interface with robotic systems lies in establishing reliable communication between the headset and the robot's middleware. To bridge the gap between the ROS ecosystem and our Unity-based application, we employ the ROS-TCP-Endpoint package [5]. This package enables direct streaming of ROS messages over TCP to the Meta Quest 3 headset, allowing for low-latency transmission of vehicle state, trajectory, and control data without requiring modifications to the existing ROS infrastructure. For spatial alignment of virtual content with the physical environment, we leverage Meta's Mixed Reality Utility Kit [4]. Specifically, we utilize its experimental QR code localization capabilities to achieve automatic track alignment (Sec. 2.2.1), enabling the virtual representation of the racetrack to be accurately overlayed onto the physical track surface.

2.2. Application features

2.2.1. Track display and alignment

Precise alignment of the virtual racetrack with the physical track surface is essential for effective mixed reality visualization. The coordinates of the track boundaries are received from ROS and rendered as a continuous lines within the application (Fig. 1). To align this virtual track with the physical environment, we implement two modes: automatic and manual.

Automatic Alignment In automatic alignment mode, the user places two QR codes at opposite corners of the bounding box spanning the physical track. Using the headset's passthrough cameras, the user scans both codes, after which the application automatically translates, rotates, and scales the virtual track to fit within the defined bounding box.

Manual Alignment For situations where QR codes are impractical or finer adjustments are needed, manual alignment mode allows direct manipulation of the track's position and rotation via the controllers. Since the track is already defined at real-world scale, scaling controls are intentionally omitted to prevent accidental distortions.

2.2.2. Car Visualization

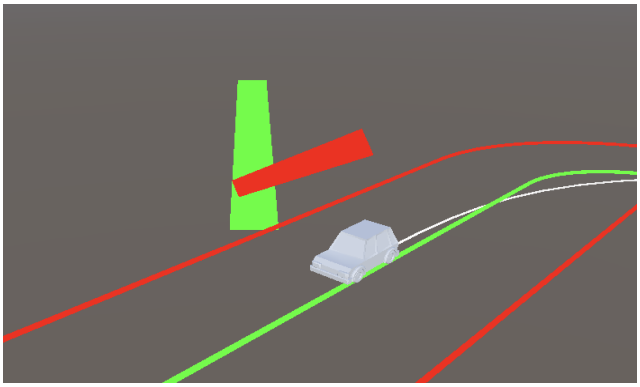
Ground truth state The most fundamental element to visualize is the ground truth position of the car, captured by the motion capture (Mocap) system. When replaying a prerecorded simulation, displaying the current vehicle position is essential. Similarly, when monitoring a live experiment, verifying the accuracy of the tracked position is valuable for debugging purposes. The ground truth position is received via ROS messages as a three-dimensional vector.

Estimated state We also implemented visualization for the estimated vehicle state. The state estimator fuses measurements from multiple sources, Mocap, IMU, and wheel encoders, to produce a filtered estimate. Since the MPC controller relies on this estimated state to compute control inputs, visualizing it is essential for debugging.

Trail visualization Displaying both states simultaneously proved visually cluttered and confusing. To address this, we implemented trail visualizations for both (Fig. 2), showing the recent path history as thin colored lines behind each car.

2.2.3. Control Input Visualization

A core function of the CRS software is computing optimal control inputs for racing. Consequently, presenting these inputs in an intuitive and informative manner was a key requirement for our framework. We implemented two visualization options and conducted A/B testing to evaluate their



(a) Car-following option



(b) Fixed HUD option

Figure 3. Control input visualisations

effectiveness (Fig. 3). The control inputs consist of steering angle and torque, both received as floating-point values from ROS.

Car-following option This option displays the control inputs attached to the moving car, allowing users to monitor both vehicle state and control inputs without shifting their attention. Torque is represented by a green vertical bar that scales according to the torque magnitude. Steering angle is indicated by an arrow positioned above the car, pointing in the steering direction relative to the vehicle's current orientation.

Fixed HUD option This option displays the control inputs at a fixed screen position. We implemented a realistic steering wheel graphic that rotates according to the steering angle, alongside a throttle bar that extends upward during acceleration and downward during braking. The bar color changes to distinguish between acceleration and braking inputs.

2.2.4. Trajectory Visualization

The final visualization component displays the two trajectories central to the CRS control loop. The MPC controller



Figure 4. Both planned and reference trajetories shown. The reference trajectory is red , while a velocity-based color gradient is used for the planned trajectory.

[3] takes the estimated state and a reference trajectory as inputs, and outputs both control commands and a planned trajectory. Visualizing both the planned and reference trajectories provides immediate feedback on whether the controller is successfully tracking the desired path (Fig. 4).

The CRS group had previously implemented trajectory visualization in their teleoperation framework, displaying trajectories as streams of color-coded points. However, during testing, we observed noticeable lag between the two trajectory visualizations. Since this functionality was implemented within the CRS software, with the visualization markers sent directly via ROS, we investigated the underlying framework. We resolved the issue by modifying the message scheduling: instead of transmitting markers individually for each trajectory type, we combined them into a single array, preventing Unity from overwriting one with the other.

3. Experiments

To assess the effectiveness of our MR visualization system for autonomous racing development, we conducted two user studies. The first study involved A/B testing with Chronos developers to compare alternative implementations of core features and tailor the system to their debugging workflows. The second study evaluated interface intuitiveness for non-expert users using the System Usability Scale (SUS) [1] assessment.

3.1. A/B Testing

3.1.1. Experiment Design

We conducted A/B testing of two features, each with two alternative implementations (Tab. 1).

3.1.2. Participants

We recruited six participants from the Chronos development team and broader robotics community:

Table 1. A/B Test Design: Evaluated Feature Alternatives

Feature	Method A	Method B
Track Alignment (Sec. 2.2.1)	QR Code Automatic + Manual fine-tuning	Pure Manual Joystick control
Control Display (Sec. 2.2.3)	Car-Following	Fixed HUD

- **Chronos developers (n=5):** Active contributors to the Chronos platform with intimate knowledge of the system architecture and typical debugging workflows
- **External participant (n=1):** Familiar with autonomous racing concepts but without direct involvement in Chronos development

Participants reported VR/MR experience levels ranging from 1 to 4 on a 5-point scale (M=3.17, SD=1.17), where 1 indicates no prior experience and 5 indicates expert level.

3.1.3. Experimental Procedure

All testing sessions were conducted in the ML Hall at ETH Zürich, where the Chronos platform's overhead camera tracking system is permanently installed. This infrastructure requirement constrained testing to a single location.

Each participant experienced both implementations of each feature, with the order of feature testing (alignment vs. visualization) randomized across participants to control for ordering effects. For alignment testing specifically, all participants tested manual alignment first, followed by QR code alignment. This order was chosen deliberately: since the QR method includes a manual fine-tuning step, testing QR first would have given participants practice with manual controls before the pure manual condition, potentially biasing timing comparisons in favor of manual alignment.

Alignment Task & Timing. Participants were asked to align the virtual track with the physical track using each method. For QR code alignment, timing began when participants received the two QR markers and included: (1) physical placement of markers at track corners, and (2) digital alignment including automatic detection and any manual fine-tuning. For manual alignment, timing began when the alignment mode was activated and ended when participants reported satisfactory alignment. All timing was recorded manually by the experimenter.

Visualization Task. Participants observed the autonomous vehicle executing laps while each visualization method was active. They were encouraged to track specific state changes (steering adjustments, acceleration/deceleration) and evaluate their ability to understand vehicle behavior in real-time.

Following completion of both conditions for both features, participants completed a comprehensive questionnaire covering:

- Likert-scale ratings (higher values indicate better performance) for intuitiveness, accuracy, ease of use, and confidence
- Preference selections with open-ended justifications
- Overall system evaluation metrics (comfort, immersion, debugging utility)
- Free-form feedback and suggestions

3.2. Non-Expert User Study

Since the visualizations and alignment methods were specifically designed for laboratory debugging workflows based on developer requirements, we did not conduct A/B testing with non-experts. Instead, we evaluated whether the interface maintained intuitiveness when used by individuals unfamiliar with Chronos.

Non-expert participants experienced the system in simulation mode rather than with the physical vehicle, as testing outside the ML Hall was not feasible. Participants observed simulated vehicle behavior with both visualization approaches (car-following and fixed HUD) and completed the System Usability Scale (SUS) [1] to assess general usability. Note that these participants did not test the track alignment feature, as it requires the physical track setup.

4. Evaluation

We present results from our two experiments, focusing on developer A/B testing results (quantitative timing data, preference patterns, and qualitative insights) followed by non-expert usability findings.

4.1. Track Alignment Performance

4.1.1. Task Completion Times

When including the complete workflow, manual alignment was significantly faster than the QR code-based approach (Figure 5). However, this comparison includes the one-time cost of physically placing QR markers, which is not representative of repeated use scenarios.

Critical insight: Once QR codes are positioned, they remain fixed on the track for subsequent sessions. Therefore, the relevant comparison for typical usage is the *digital alignment phase only*. When excluding physical QR placement and comparing only alignment time, QR-assisted alignment and manual alignment showed *no statistically significant difference* ($p = 0.13$), suggesting comparable performance for repeated use.

The QR method comprises two distinct phases: physical marker placement (performed once during initial setup) and digital alignment with automatic detection plus manual fine-tuning (repeated for each session). The high variability in both QR placement (CV=70.3%) and QR alignment (CV=53.3%) contrasts with manual alignment's consistency (CV=27.9%), suggesting performance depends heavily on user skill and marker placement quality.

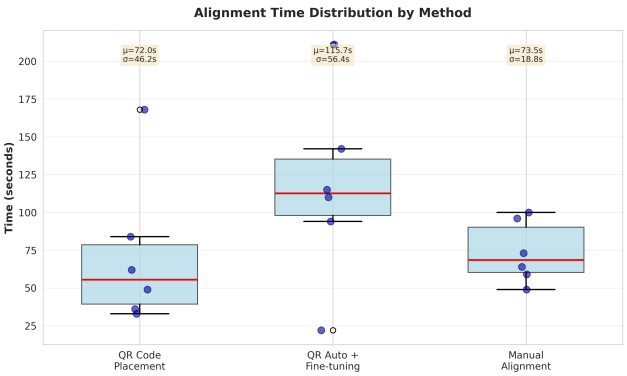


Figure 5. Time distribution for alignment methods showing QR code placement (left), QR automatic alignment with fine-tuning (center), and pure manual alignment (right).

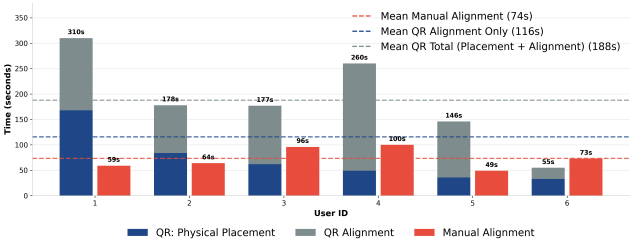


Figure 6. Individual user performance for QR-based track alignment (split into physical QR code placement and alignment) versus manual track alignment. Dotted lines indicate mean values for each method

Figure 6 shows per-user performance. Five of six participants completed faster with manual method when including setup, but User 6 achieved remarkably fast QR performance (55s total, 22s alignment only), demonstrating that optimal QR placement enables efficient workflows.

4.1.2. User Preferences and Ratings

User preferences revealed nuanced attitudes toward each method. Four participants (67%) preferred manual alignment, one preferred QR codes, and one expressed no preference. Interestingly, this preference pattern emerged despite the lack of significant timing difference in the digital alignment phase, suggesting that factors beyond speed influenced preference. Alignment method ratings showed that users appreciated different aspects of each approach (Tab. 2).

Users found the QR concept intuitive (rating higher than manual) but were dissatisfied with execution accuracy, particularly for scale calibration. Manual alignment's higher precision rating reflects users' ability to iteratively refine alignment to their satisfaction. However, several participants noted that control mappings became confusing when they changed their physical viewing perspective.

Table 2. User ratings for track alignment methods. Higher values indicate better performance (scale: 1 = low, 5 = high).

Method	Metric	Rating (M $\pm$ SD)
QR Code	Intuitiveness	$\mathbf{4.00 \pm 0.89}$
	Perceived Accuracy	3.33 ± 0.82
Manual	Intuitiveness	3.67 ± 0.82
	Perceived Accuracy	$\mathbf{4.17 \pm 0.75}$

Impact of one-time setup: The preference for manual alignment likely reflects participants' experience during testing, where each person performed the complete QR setup process (including marker placement) only once. In real-world deployment scenarios where QR codes remain fixed on the track across multiple sessions, the initial placement overhead becomes amortized. The comparable digital alignment times ($p = 0.13$) suggest that for repeated use, the QR method may be more competitive than preferences indicate, particularly once users develop proficiency with marker placement.

4.1.3. Qualitative Insights: The Hybrid Approach

Open-ended responses revealed unanimous convergence on a hybrid workflow: using QR codes for coarse initialization followed by manual fine-tuning. All six participants independently advocated for this combined approach, with consistent themes emerging across responses (see Appendix A for representative quotes).

The feedback indicates that developers value QR initialization primarily for accurate position and orientation alignment, which reduces the search space for manual refinement. However, scale calibration emerged as a consistent limitation of the automatic method, requiring manual correction in most cases. Participants found manual adjustment intuitive and precise, though several noted that control mappings became confusing when changing viewing perspective.

Following the user study, we implemented several refinements based on this feedback: (1) visual bounding box overlay showing the rectangle the QR markers span to assist with placement accuracy and (2) orientation independent joystick controls.

4.2. Control Input Visualization

4.2.1. Preference Patterns

Visualization preference data showed a clear majority favoring the fixed HUD display (Visualization B). Four participants (67%) preferred the HUD overall, while two preferred the car-following display. When asked specifically about information processing speed, preferences were more divided: three participants cited HUD as faster, two cited car-following as faster, and one reported similar performance.

Table 3. User ratings for fixed HUD component design. (scale: 1 = low, 5 = high).

Component	Intuitiveness (M $\pm$ SD)
Steering Wheel	$\mathbf{5.00 \pm 0.00}$
Throttle Bar	4.83 ± 0.41

Table 4. User ratings for tracking control inputs during vehicle motion (scale: 1 = low, 5 = high).

Metric	Rating (M $\pm$ SD)
Ease of Tracking Steering	3.00 ± 1.67
Ease of Tracking Throttle	2.83 ± 1.60
Confidence in Reading Values	3.83 ± 0.75

Component-level ratings revealed that both visualizations were well-received for their individual elements (Table 3). The steering wheel representation achieved perfect scores from all participants, while the throttle bar was also highly rated.

However, ratings for tracking ability during vehicle motion exposed a challenge (Table 4). This discrepancy—high representation intuitiveness but moderate tracking ability—suggests that while the visualization designs themselves are clear, following rapid state changes during dynamic vehicle behavior remains cognitively demanding.

4.2.2. Context-Dependent Utility

Qualitative feedback revealed that preference was context-dependent rather than absolute (see Appendix B for representative quotes). The fixed HUD was valued for providing detailed, stable information with highly visible representations, particularly the steering wheel display which received perfect ratings. However, participants noted a key limitation: in mixed reality environments where users naturally track the moving vehicle by turning their heads, fixed world-space HUD elements may fall outside the field of view. One participant suggested head-locked HUD elements (attached to headset orientation rather than world coordinates) as a potential solution.

Conversely, the car-following display was valued for spatially co-locating control information with the focus of attention, eliminating the need to shift gaze between the vehicle and information displays. This design proved particularly valuable for attention-focused observation tasks where understanding the relationship between control inputs and vehicle motion is paramount.

This feedback suggests that rather than one visualization being universally superior, the optimal choice depends on the debugging task: HUD for stable monitoring of precise values, car-following for attention-focused observation of behavior. As one participant summarized: "keep both options available, each has a different use."

Table 5. User ratings for overall MR interface experience (scale: 1 = low, 5 = high).

Metric	Rating (M $\pm$ SD)
Enhanced Understanding of Car Behavior	4.33 ± 0.82
Visual Comfort	4.33 ± 0.52
Immersiveness	4.17 ± 0.75
Overall Intuitiveness	3.67 ± 1.03
Debugging/Monitoring Utility	3.83 ± 1.60

4.3. Overall System Evaluation

Participants rated the overall MR interface highly across multiple dimensions (Table 5). The system achieved strong ratings for behavior understanding and visual comfort, with good immersiveness. Overall intuitiveness and debugging utility showed more variance, with debugging utility displaying the highest standard deviation, likely reflecting individual differences in how developers prefer to debug.

Only one participant reported mild motion sickness, and five participants reported no distraction or occlusion issues from UI elements. However, critical state change identification showed mixed results, with responses ranging from "Very Slow" to "Fast" (2 Fast, 2 Moderate, 1 Slow, 1 Very Slow), indicating potential areas for improvement in real-time feedback mechanisms.

These results demonstrate that the MR visualization system successfully enhances developer understanding of autonomous vehicle behavior in a comfortable, immersive environment. The variability in debugging utility ratings and state change identification suggests that future iterations should focus on customizable visualization options that accommodate individual debugging preferences and improve temporal dynamics for tracking rapid state changes.

4.4. Non-Expert Usability Results

Six non-expert participants (age range: 21-64, M=37.5 years) with varying VR experience levels (1 = low, 5 = high) evaluated the interface in simulation mode. The system achieved a mean SUS score of 70.0 (SD=16.6, range: 40-90), indicating good usability and performing above the average benchmark of 68. Two participants achieved excellent scores (>80), while one participant reported difficulty (SUS=40), reflecting individual differences in VR familiarity and technical background. The results suggest the interface maintains reasonable usability for users unfamiliar with autonomous racing contexts, though optimal experience may require prior VR exposure.

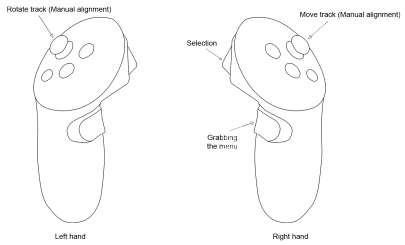


Figure 7. Controls Help menu displaying the Meta Quest controller layout and button mappings to assist new users.



Figure 8. Overview of the menu interface showing (from left to right) the CRS Menu, Network Settings, Application Status, Controls Help and a toggle to hide the menu.

5. User-Driven Refinements

The following section shows improvements that have been made to the application after performing the aforementioned user studies.

5.1. Menu Design

User testing revealed that participants unfamiliar with VR applications struggled with the typical control schemes and had difficulty navigating the initial menu structure. Based on this feedback, we redesigned the interface by removing non-essential functionalities and focusing exclusively on vehicle visualization features (Fig. 8). The resulting menu is organized into five sections:

- **CRS Menu:** Controls all visualization-related features, including toggles for individual visual elements and access to track alignment functions.
- **Network Settings:** Allows the user to specify the IP address of the ROS TCP stream for incoming vehicle data.
- **Application Status:** Displays the current connection state of the TCP stream and provides debug output for troubleshooting.
- **Controls Help:** Shows an overview of the application controls to assist new users (Fig. 7).
- **Hide Menu:** Hides the menu panel.

We included the option to close the menu panel for users who prefer a cleaner view. When the menu is hidden, only a single button remains visible to restore it, leaving the scene unobstructed.



Figure 9. Tooltip displaying a button's functionality when the user hovers over it.

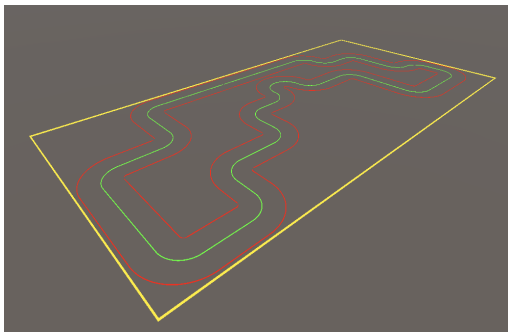


Figure 10. Virtual track with bounding box visualization (yellow) for alignment verification. The track boundary (red) corners do not extend to the bounding box corners, illustrating why the bounding box aids in verifying correct placement.

5.2. Tooltips

To further assist users unfamiliar with the application, we implemented hover tooltips for all interactive buttons (Fig. 9). When the user hovers over a control element, a brief description of its functionality appears. This provides contextual guidance without cluttering the interface, reducing the learning curve for new users.

5.3. Track bounding box

To further assist with track alignment, we added an option to display the bounding box of the track. Since QR codes are placed at opposite corners of this bounding box, they do not coincide with the track's outermost points, making it difficult to visually verify alignment accuracy. The bounding box allows users to quickly identify errors in scale or rotation by comparing the virtual bounds to the physical track edges. This is particularly useful in automatic alignment mode, but also aids manual adjustments (Fig. 10).

5.4. Extensibility

A key requirement from the CRS team was to ensure that our application can be maintained and extended by future developers. We retained the existing architecture from

the teleoperation toolkit [6], keeping visualization features modular to allow new components to be added independently. Additionally, we provided setup documentation covering the ROS environment configuration, TCP connection, and Unity project setup, enabling researchers in the lab to reproduce the development environment and build upon our work.

6. Discussion

6.1. Limitations

The automatic track alignment relies on experimental QR code localization features from Meta's Mixed Reality Utility Kit [4]. Since experimental features cannot be included in published applications, releasing the app on the Meta Quest store would require a custom implementation of QR code detection and spatial localization. However, this limitation has minimal impact on the intended use case, as the CRS lab can run the application directly as a development build without requiring store publication. A custom implementation was therefore deemed out of scope for the current project.

6.2. Future Work

Developer feedback identified several key areas for future enhancement.

For track alignment, participants preferred combining QR code initialization with manual refinement in the current implementation. However, this workflow emerged primarily due to limitations in automatic scale calibration accuracy rather than fundamental preference for manual work. Future development should focus on improving the QR-based alignment to function reliably without manual adjustment.

Visualization preferences revealed task-dependent needs. Participants suggested head-locked HUD elements (fixed to headset orientation rather than world space) to address visibility issues when tracking the moving vehicle.

In conclusion, future development should prioritize customizable interfaces allowing developers to select their preferred visualization options based on their specific debugging tasks.

A. User Feedback: Track Alignment

Participants unanimously advocated for combining QR code initialization with manual refinement. Representative quotes:

User 1

> Actually starting with QR and micro-adjust with manual was the best. The auto alignment saves time putting the projected track close to real track (scale, position, orientation). Manual adjustment is intuitive, if the controls are issued with the same initial perspective.

User 2

> A mix of both seems like the best combo: Use the automatic alignment to snap into close alignment and then manually adjust to fine-tune in case the QR codes were placed slightly outside/inside the true bounding box.

User 3

> Only the scale was a bit off, and had to be adjusted manually after the auto alignment. But the positioning seemed very accurate.

B. User Feedback: Control Input Visualization

Participants described context-dependent preferences for visualization methods:

User 1 - Fixed HUD advantages

> The fixed position values are more detailed, and the steering wheel rendering is more visible compared to the car following bar.

User 1 - Head-locked suggestion

> But it's mixed-reality, I tend to track the car by moving my head. So I usually don't see the fixed position info. IMO, it would be even better to have the HUD data fixed to the 'headset' (instead of either fixed in space or attached to the car).

User 2 - Car-following advantages

> I tended to naturally gaze at the car, this puts the information right next to where things are happening.

User 4 - Complementary use cases

> Control input in the HUD is intuitive and game like. It is hard to keep up with the control input and the car at the same time. That is why it makes sense to have the parameters follow the car. I would say keep both options available, each has a different use.

References

[1] John Brooke. Sus: A quick and dirty usability scale. *Usability Evaluation in Industry*, 1996. 3, 4

[2] Andrea Carron, Sabrina Bodmer, Lukas Vogel, René Zurbrügg, David Helm, Rahel Rickenbach, Simon Muntwiler, Jerome Sieber, and Melanie N. Zeilinger. Chronos and crs: Design of a miniature car-like robot and a software framework for single and multi-agent robotics and control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, page 1371–1378. IEEE, 2023. 1, 2

[3] Carlos E. García, David M. Prett, and Manfred Morari. Model predictive control: Theory and practice—a survey. *Automatica*, 25(3):335–348, 1989. 3

[4] Meta Platforms, Inc. Mixed Reality Utility Kit (MRUK), 2023. Accessed: 2026-01-08. 2, 8

[5] Unity Technologies. ROS-TCP-Endpoint: ROS package for accepting ROS messages from Unity, 2022. Accessed: 2026-01-08. 2

[6] Maximum Wilder-Smith, Vaishakh Patil, and Marco Hutter. Radiance fields for robotic teleoperation. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 13861–13868, 2024. 1, 2, 8